

High Performance Programming

Lecture 7

”Shared Memory Programming”

Lecturer: Stefan Nordborg Eriksen

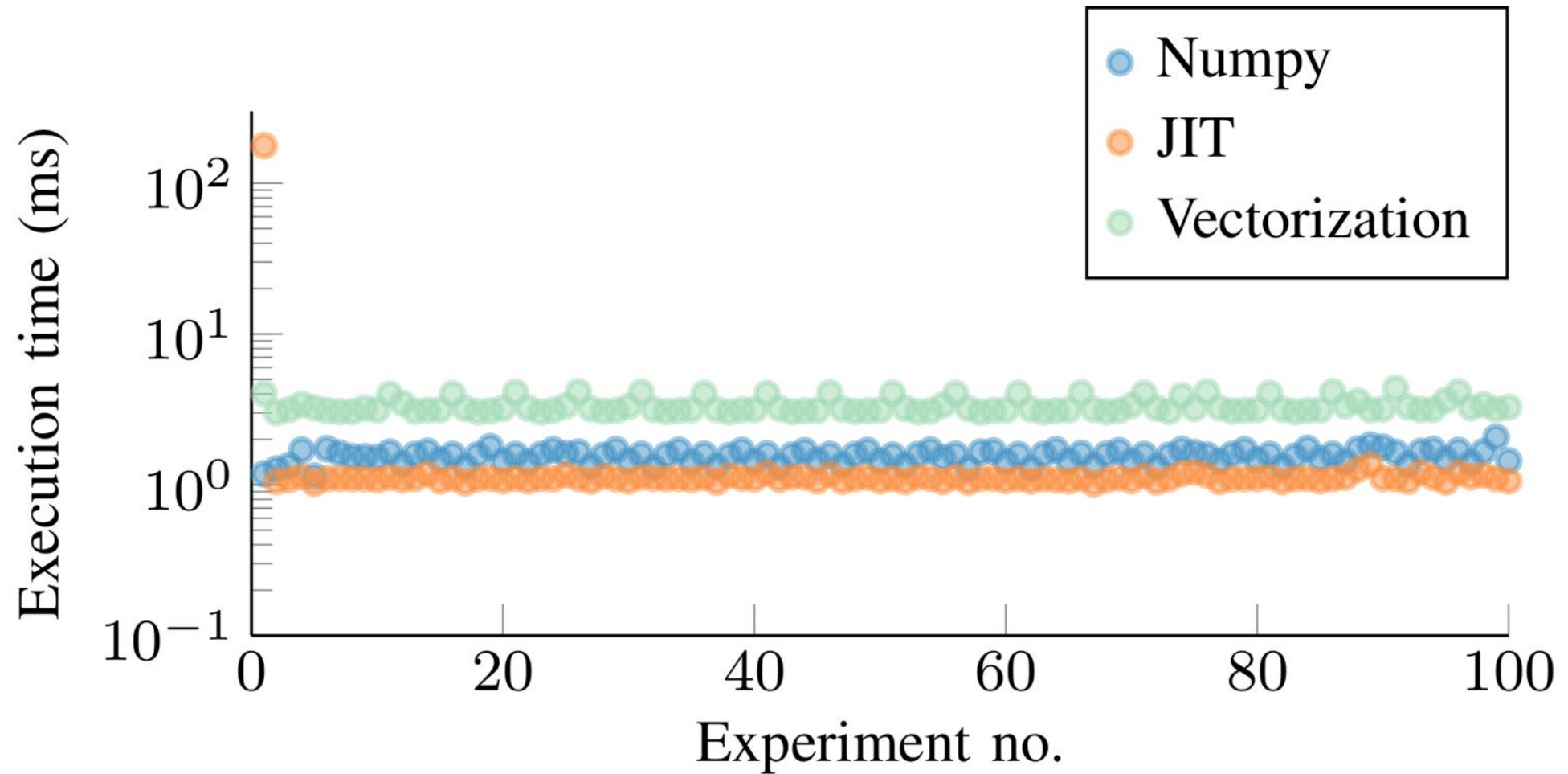


AALBORG
UNIVERSITET

Contents

1. Basics of shared memory programming
2. Multithreading vs. multiprocessing
3. Automated parallel execution and performance results
4. Applications and examples

Execution times from last lecture with N=1000



1. Basics of shared memory programming

Shared memory basics

Communication is implicit due to the sharing of memory among processes

Privacy is not a problem

No need to enforce data protection: we assume the processes cooperate

We do however **need** instructions to avoid excessive communication

Focus: Constructs for concurrency and synchronization with minimum overhead

Portability: tasks run in parallel if possible on similar platform. Otherwise, they run sequentially

Latency hiding: scheduler optimizes tasks execution to avoid idleness

More synchronization operations available

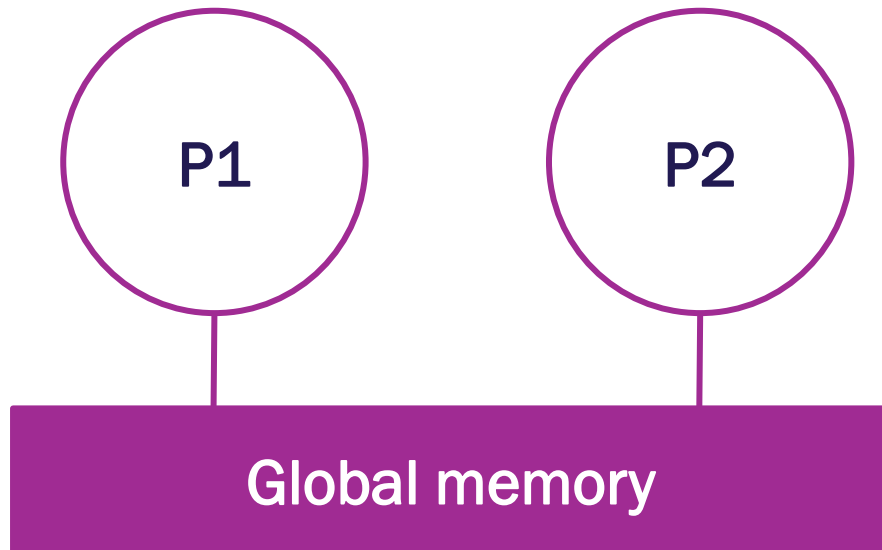
Shared memory architectures are not scalable

For bigger architectures, you would go for distributed memory

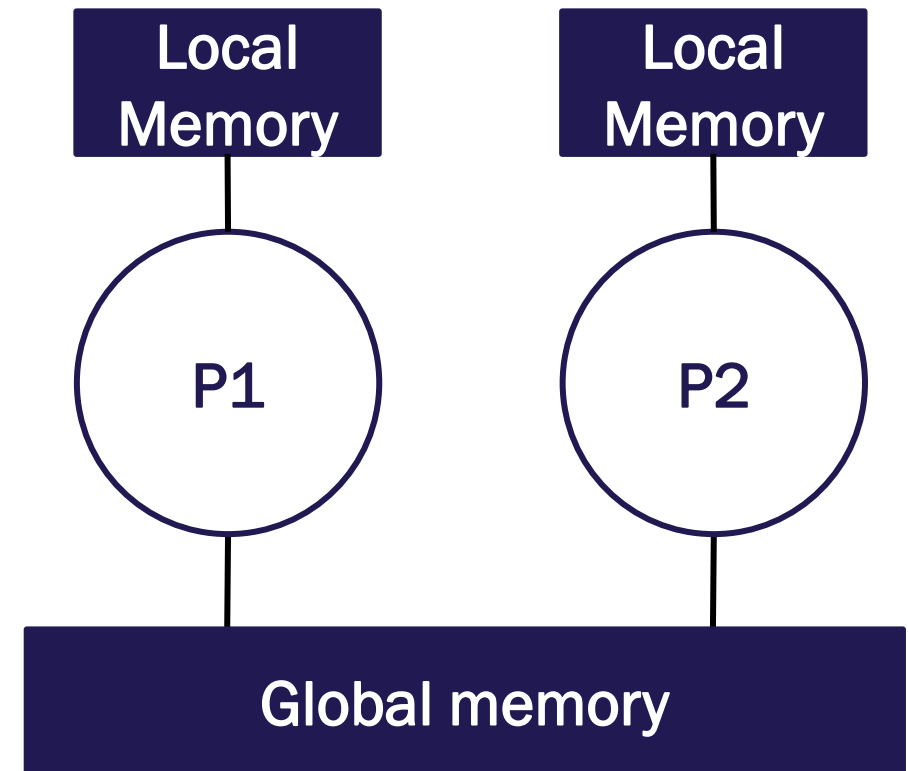
The paradigm of shared memory access

Assumption: Fully shared memory

Overlooks some practical aspects



Reality: Caches, registers, and the stack?



Single-Instruction Multiple Data (SIMD) model

Same instruction to be executed with different parts of the data

Performance-oriented objective with p parallel tasks:

Speedup $S(n, p) = \frac{T_s^*(n)}{T_p(n)}$

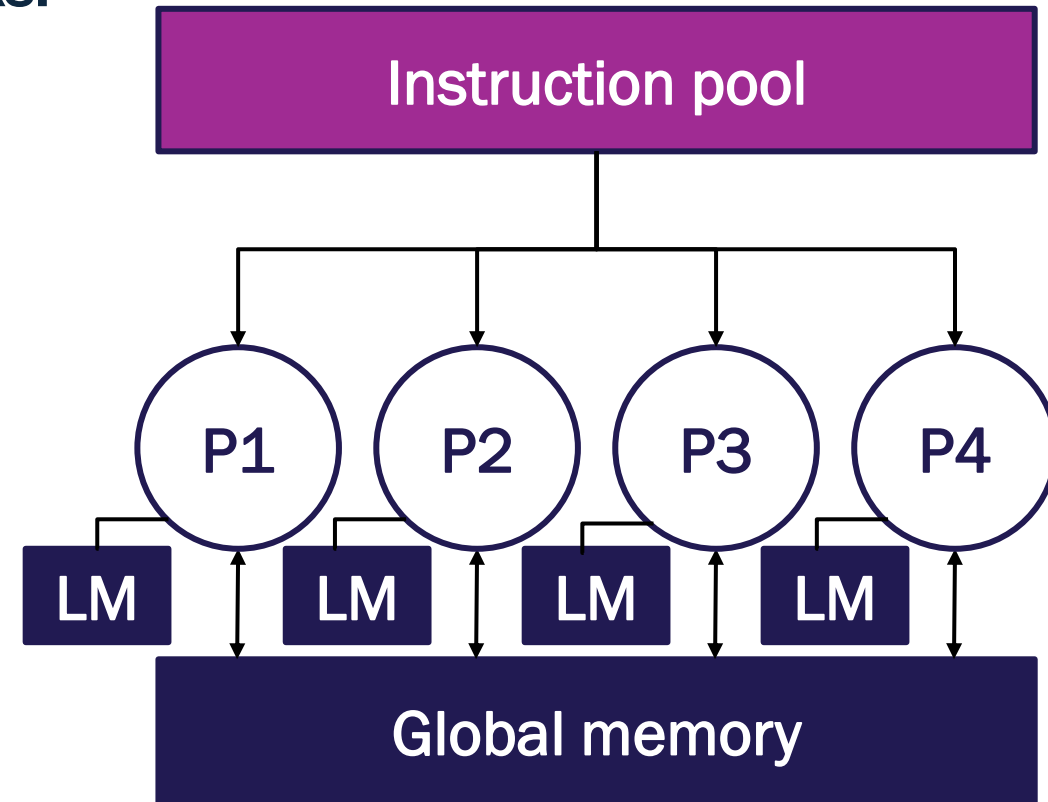
Avoid data dependencies; reduces potential gains

Non-uniform Memory Access (NUMA)

Faster access to own processor data

Vectorized operations in built-in functions

Highly efficient, several instructions per clock cycle



Multiple Instructions Multiple Data (MIMD) model

Different instruction to be executed in different processing units

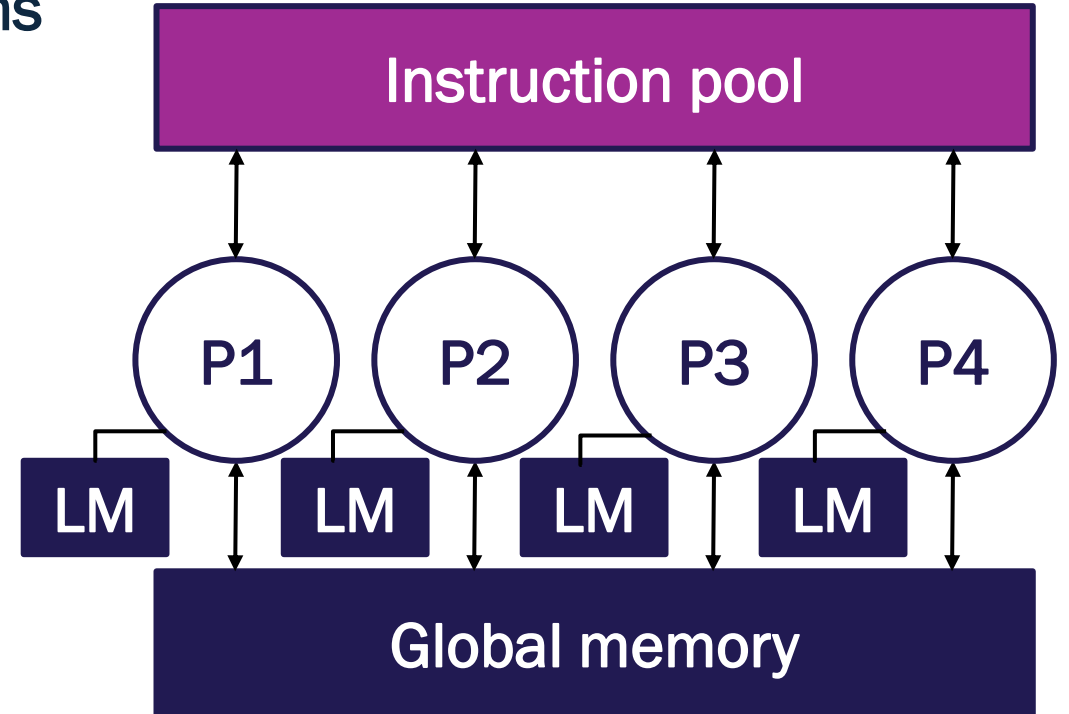
Multiple Instructions Single Data (MISD) is a special case

General approach, applicable to complex problems

Especially relevant for real-time programming

Useful for **latency hiding**

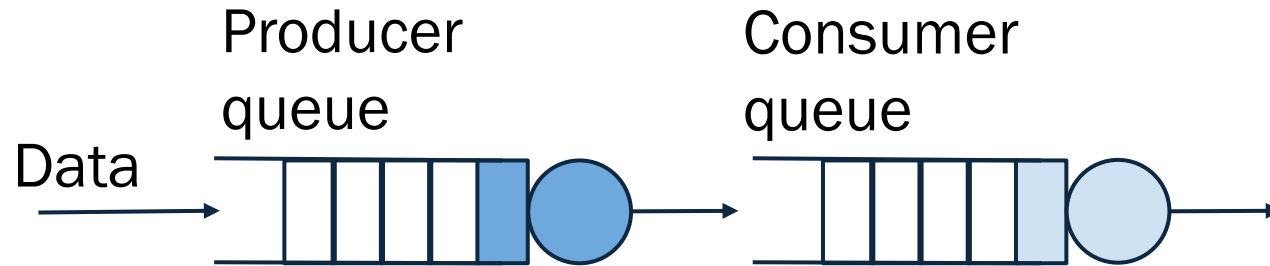
Avoid idle periods and keep the CPU busy



Producer-consumer queues

The **producer** generates tasks at its own pace

The **consumer** picks up tasks and executes them



There are many applications of producer-consumer queues:

- **Communication systems**
- **User interface**
- Media streaming services
- Gaming

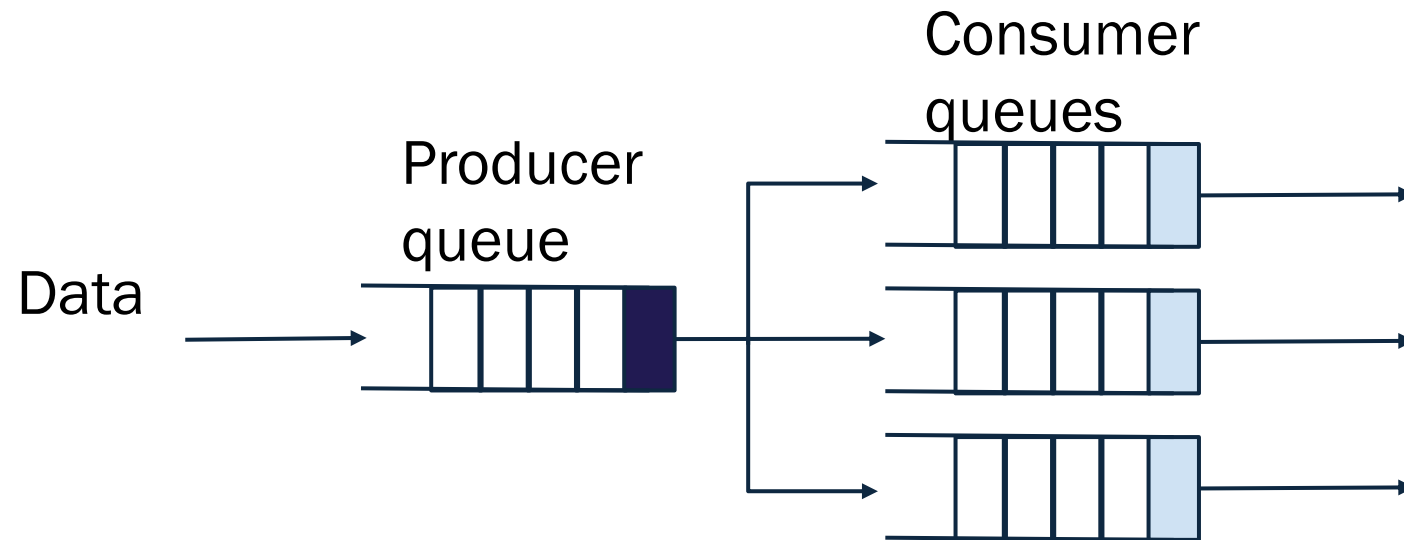
Producer-consumer queues

Trivial example: Use of a variable `task_available`: 1 if there is a task, 0 if empty

Allow multiple threads to claim a task when ready

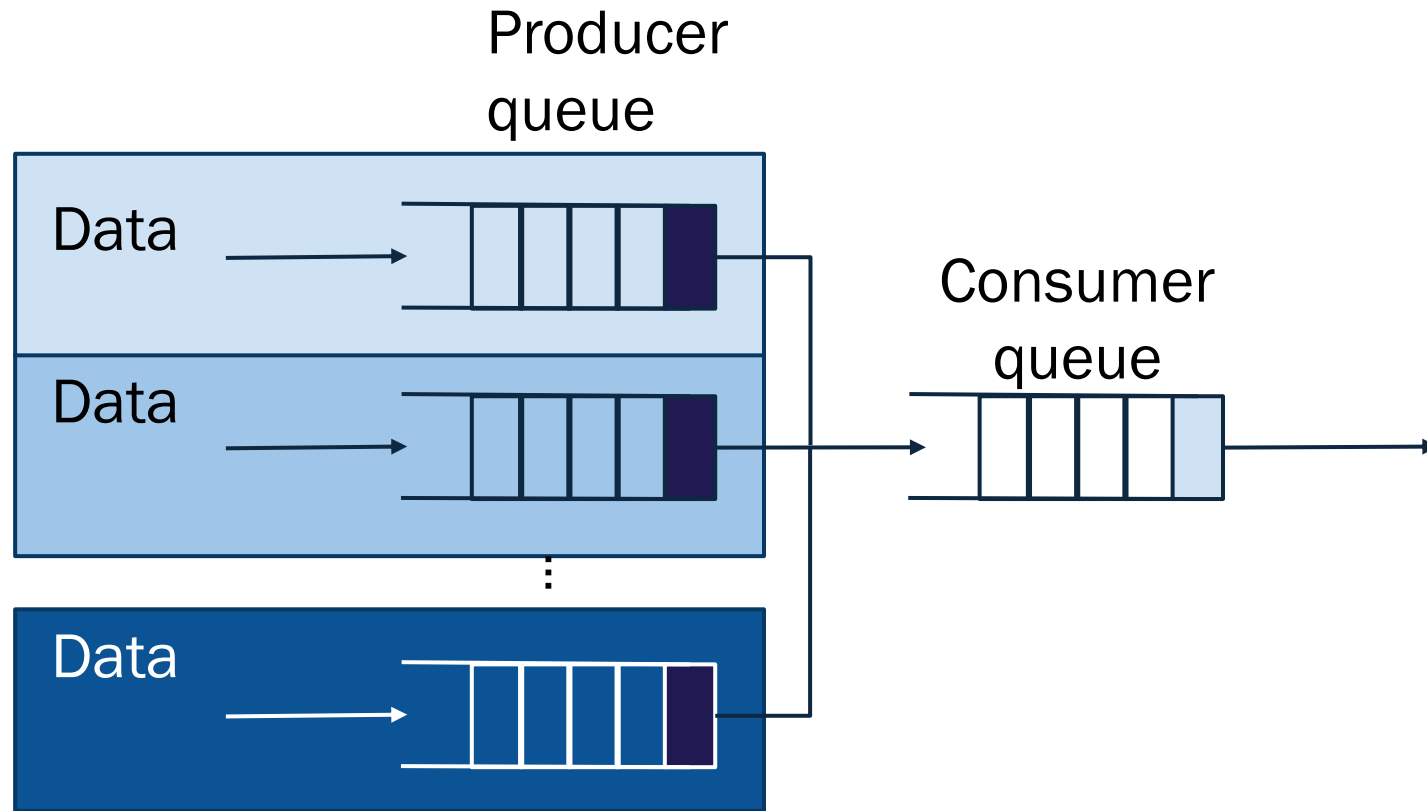
But protect the task queue so only one process claims and executes the task

This avoids doing extra work and having data inconsistencies



Producer-consumer queues

Where can we exploit parallelism?



2. Multithreading vs. multiprocessing

Maximum speedup according to Amdahl's law

Parallel programming has its limits

A speed(up) limit that we cannot exceed

Maximum speedup is p , the no. of processors

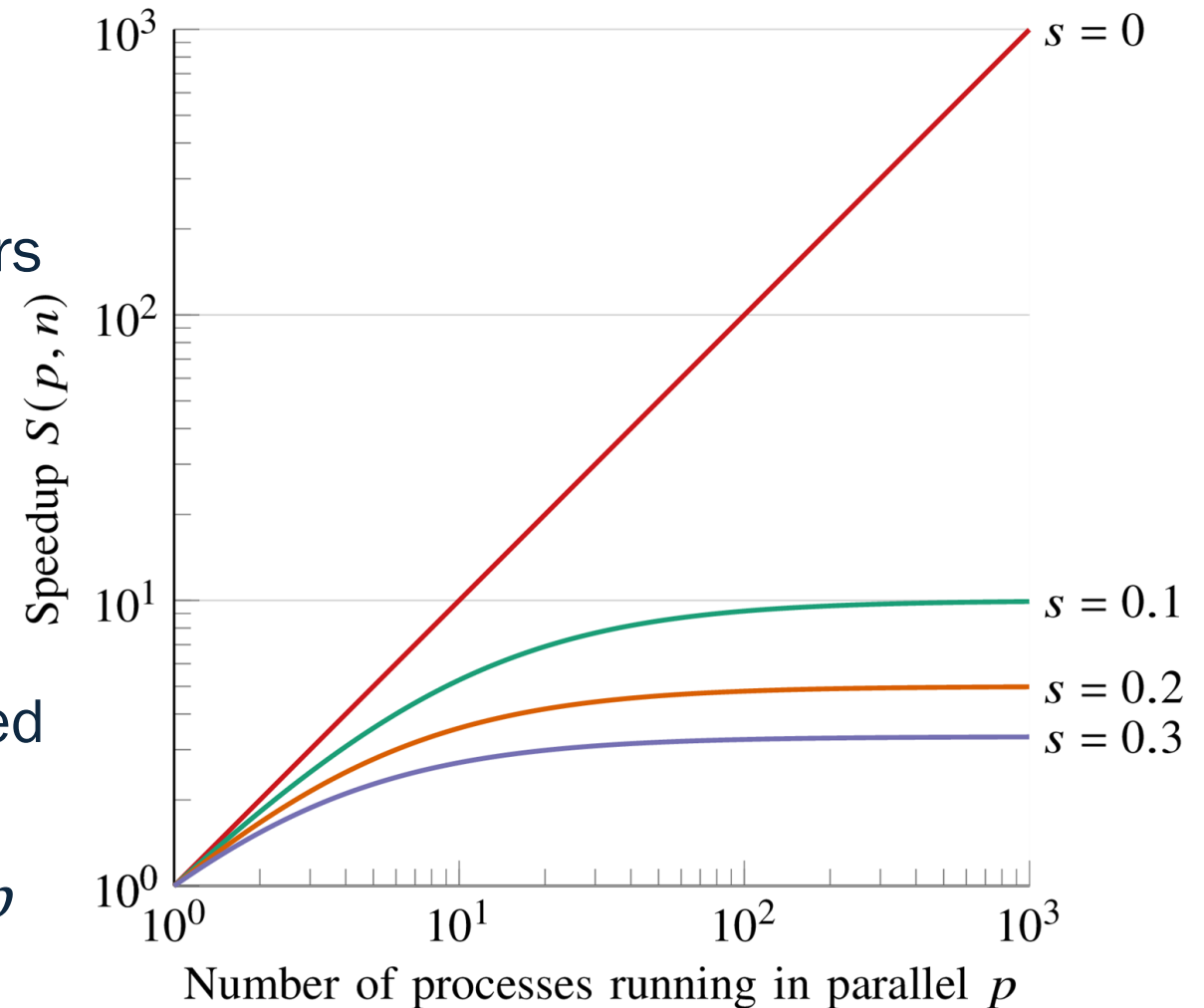
This is assuming

- 1.No communication overhead
- 2.A problem of fixed size n
- 3.Full parallelism

A fraction s of the code cannot be parallelized

s depends on the problem size

Communication overhead increases with p



What is a thread?

Separate flow of execution

In most programming languages, threads execute concurrently

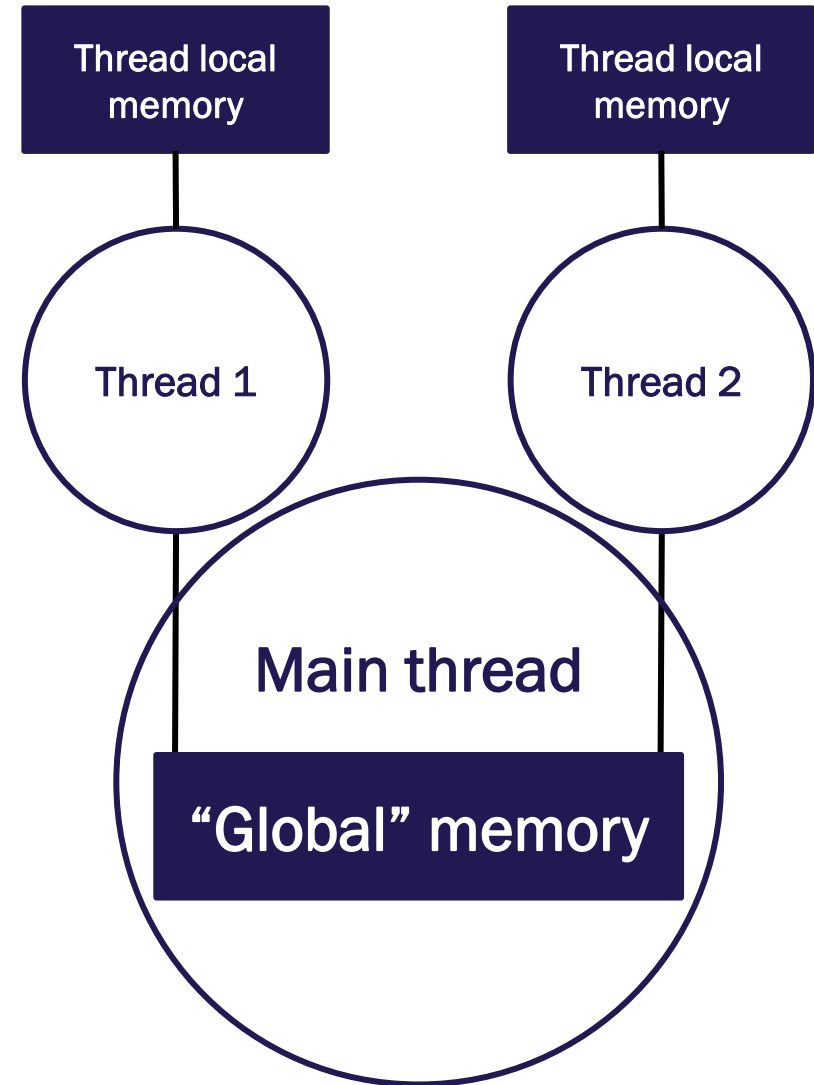
“Global” data is available to all the threads

But there is still local memory:

- Cache
- Registers
- Stack

We need to ensure data consistency

Focus: Constructs for concurrency and synchronization with minimum overhead



Thread execution diagram (UML)

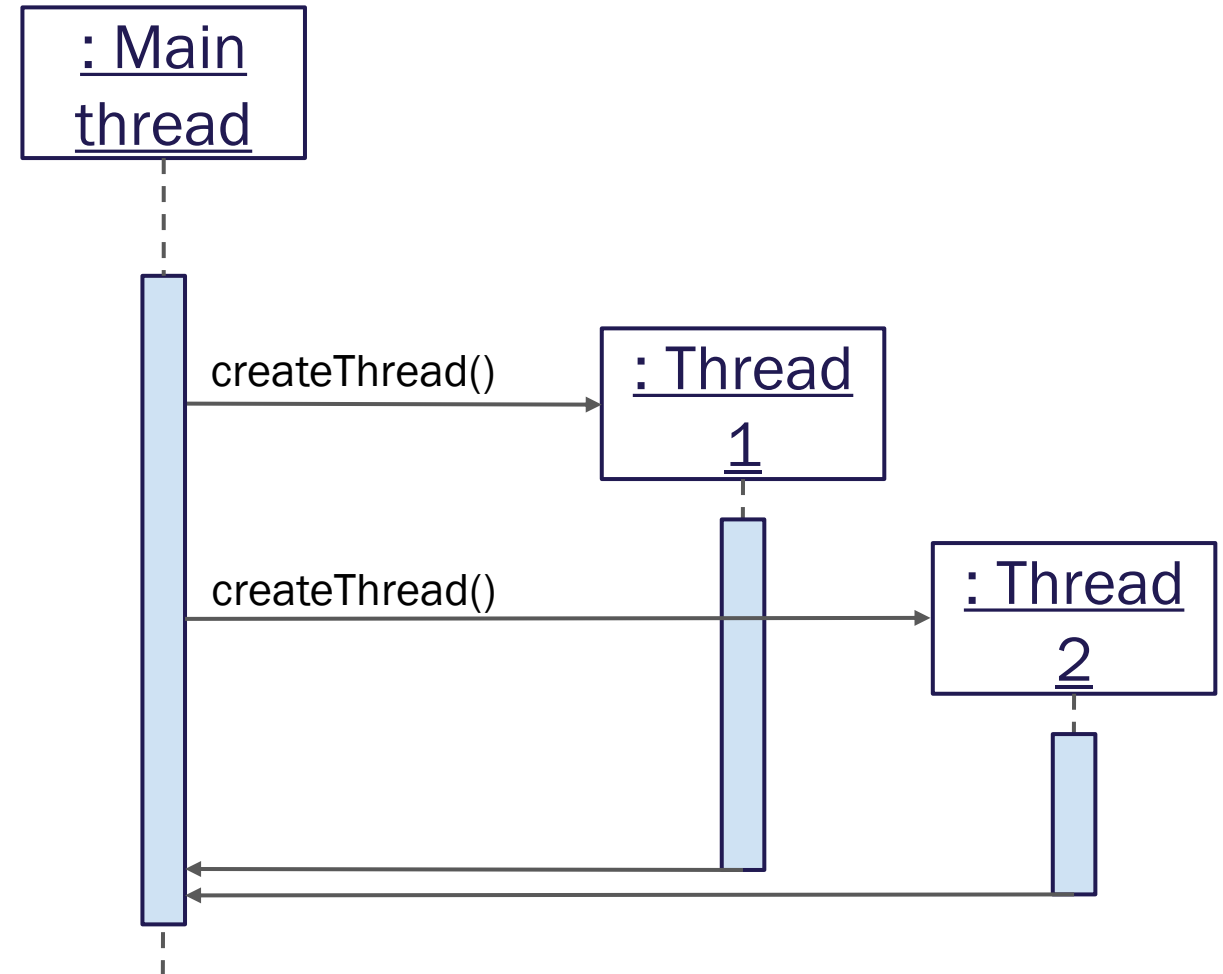
There is a main thread

Your main program

It is in charge of creating more threads

Must wait for them to finish

Must coordinate access to memory



Threading in Python

“In CPython, due to the Global Interpreter Lock, **only one thread can execute Python code at once**... However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.”

—<https://docs.python.org/3/library/threading.html>

Useful for: Tasks that must wait for external events

Not good for: Tasks that must execute heavy computations concurrently

Multithreaded programming basics

A thread can execute any **function** in your code

1.Create a thread: you define the function and the arguments to pass to it

```
f = threading.Thread(target=threaded_function, args=(index,))
```

1.Start the thread: `f.start()` initiates the activity in the thread

2.Join the thread: `f.join()` the main thread waits until the thread terminates

You can see the threads running with `print(threading.enumerate())`

Thread-local variables can be defined. To create a variable `x` that is different for each thread:

```
threadlocal = threading.local()
```

```
threadlocal.x = val
```

What is a process?

Instance of a program executed by one or multiple threads

No direct communication between processes

Specific instructions must be used to pass data

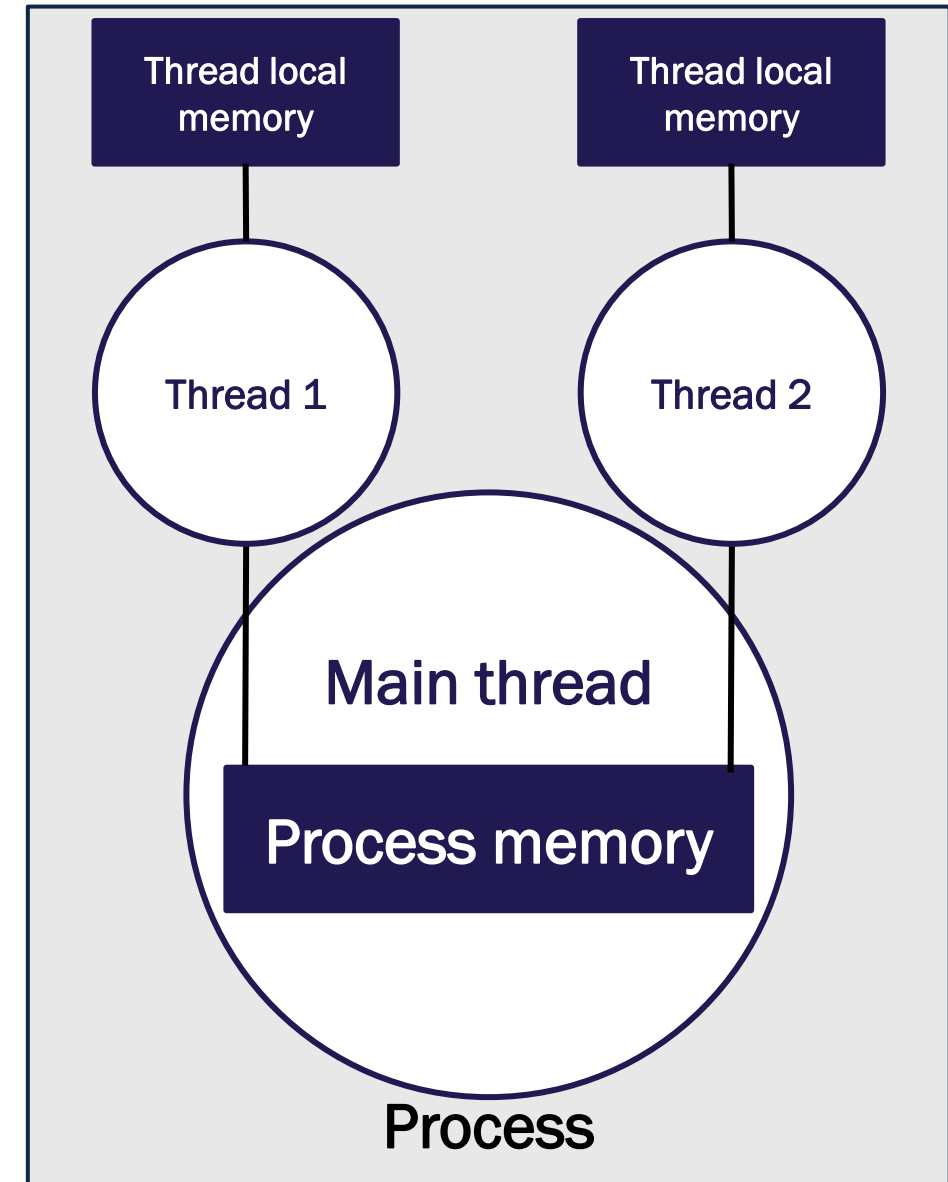
Processes have an ID: PID

Check the running processes and threads

In Linux: Run “top -H” or “htop” in terminal

Windows: Open task manager and click Processes tab

How many threads and processes are running?



Multiprocessing in Python

“The multiprocessing package offers both local and remote concurrency, effectively side-stepping the Global Interpreter Lock by using subprocesses instead of threads.”

—<https://docs.python.org/3/library/multiprocessing.html>

Good for: Tasks that must execute heavy computations concurrently

Avoid use when: data must be frequently shared between tasks

Multiprocessing synchronization and communication

Includes equivalents for all synchronization primitives from threading

Locks, Condition(): same as in threading

Data must be passed to the processes

Plus some options to pass values

Value: `n = mp.Value('d', 0.0)`
#Creates a shared variable of type double set to 0

Array: `arr = mp.Array('i', range(n))`
#Creates a shared integer array set to [0, 1, 2,..., n]

Queue: `q = mp.Queue()`
Creates an empty queue to save return values

The costs of threading and multiprocessing

- 1.Starting and stopping tasks
- 2.Task (i.e., context) switching may require to load more data into memory
- 3.Synchronization and communication between tasks
- 4.Tasks executed in parallel need separate storage

The commandments of threading and multiprocessing

1. Do not spawn a new task if its execution takes less than its creation
 2. Declare task-specific variables when used by multiple tasks in parallel
 3. Avoid accessing and modifying “global” variables directly
 4. Parallel execution can be used to speed-up SIMD problems and also MIMD (latency hiding)
 5. Balance the load among the tasks
- Rule of thumb: Run at most one thread per “core”

Examples

Pitfalls of parallel programming

Mostly applicable to threading

- 1.**Race conditions:** Several tasks compete to access/modify the same data
- 2.**Starvation:** A task is denied access to a resource for a long time
- 3.**Deadlock:** The task comes to a standstill as a result of locked access to memory
- 4.**Livelock:** The task keeps running in a loop and cannot progress

Locks (with threading)

Mutual exclusion (mutex) locks: Class used to coordinate access

Synchronization primitive that is either “locked” or “unlocked.”

Changes to a mutex lock are atomic (happen in a single clock cycle)

Define variable lock:

```
_lock = threading.Lock()
```

Acquire and release lock:

```
with _lock:
```

Check if the lock is locked:

```
_lock.locked()
```

Acquire the lock:

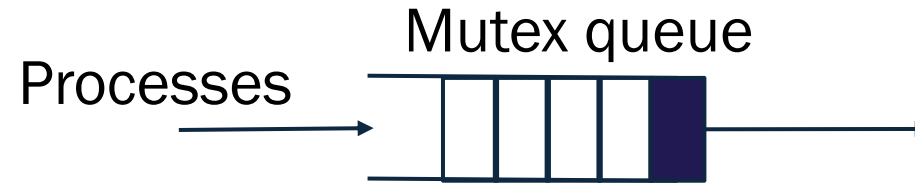
```
_lock.acquire()
```

Release the lock:

```
_lock.release()
```

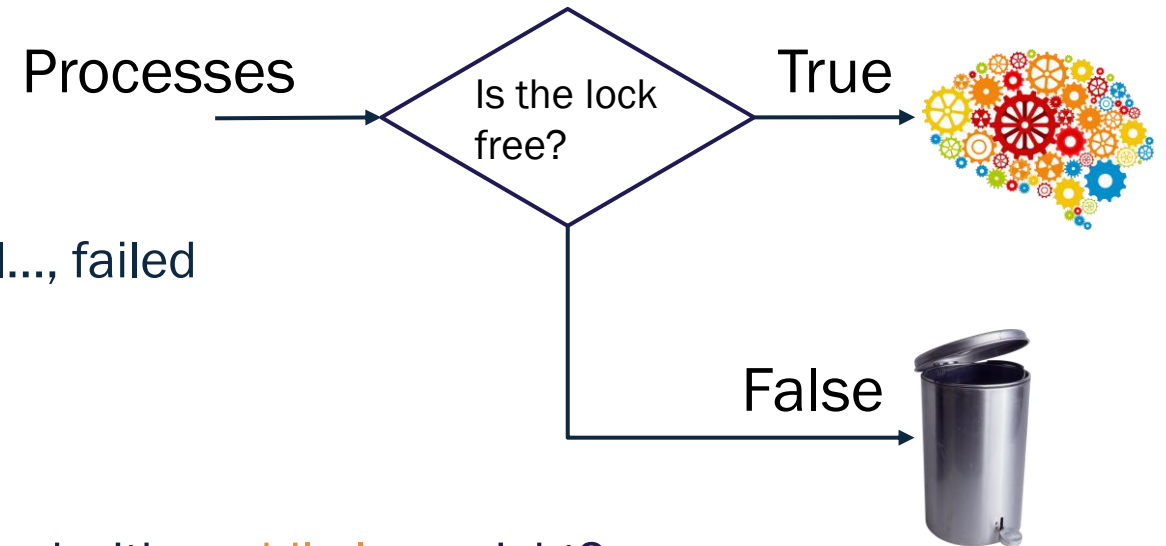
Problem: locking introduces overhead

If `task_available` is locked with `mutex.acquire()`; the process cannot continue.



Instead, use `mutex.acquire(blocking=False)`:

Attempts to lock: **True** if successful and **False** if, well..., failed



Execution of a specific task may fail, but can be solved with a **while loop**, right?

Continuous polling: the “are we there yet?” problem

How frequent should the status of the lock be checked? How many times is too much?

Are we there yet?

Are we there yet?

Are we there yet?

Are we there yet?...

Checking the lock consumes resources

Solution: condition variables



Credit: The Simpsons

Condition variables

Synchronization primitive

Always has a mutex lock associated with it

If a pre-specified condition is True, a notification is sent to specific threads

These resume their execution

```
cv = threading.Condition()      # Declare CV
with cv:                        # Locks the access associated to the CV
    cv.wait()                   # Wait until the CV is unlocked
    cv.notify()                 # Notifies one thread that the CV is unlocked
    cv.notifyAll()              # Notifies all threads that the CV is unlocked
```

Basically, an event-based approach!

3. Automated parallel execution

Pool executors

Tasks can be automatically mapped to processes and threads

Use **concurrent.futures** package

Uses a Process or Thread PoolExecutor

```
with concurrent.futures.ProcessPoolExecutor(max_workers=n_cores) as executor:  
    executor.map(function, arguments)
```

The length of arguments determines the number of processes or threads

Useful for simple tasks

Check out for further examples:

<https://www.packetswitch.co.uk/what-is-concurrent-futures-and-how-can-it-boost-your-python-performance/>

Numba JIT

Numba has an option to enable parallel execution

From our vector-matrix multiplication example from last week

```
@jit(nopython=True, parallel=False)
def numba_jit_mult(x, A):
```

...

Just(?)

1.Change parallel = True

2.Change range to prange in the loops to parallelize

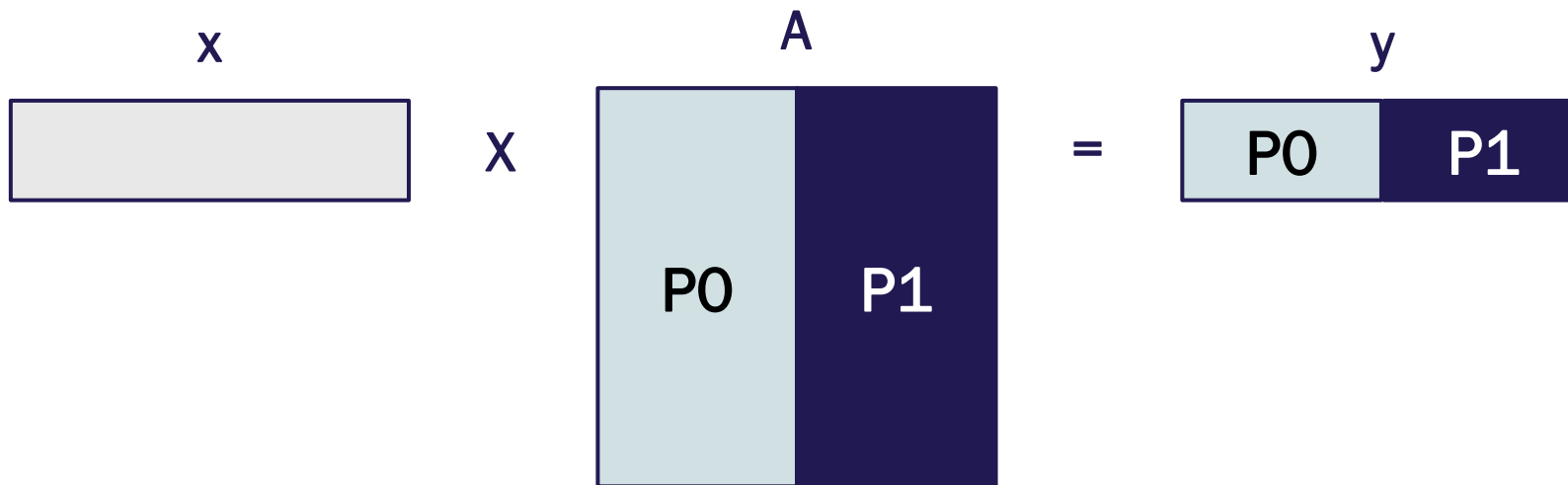
See [here](#) for some tips



4. Applications and examples

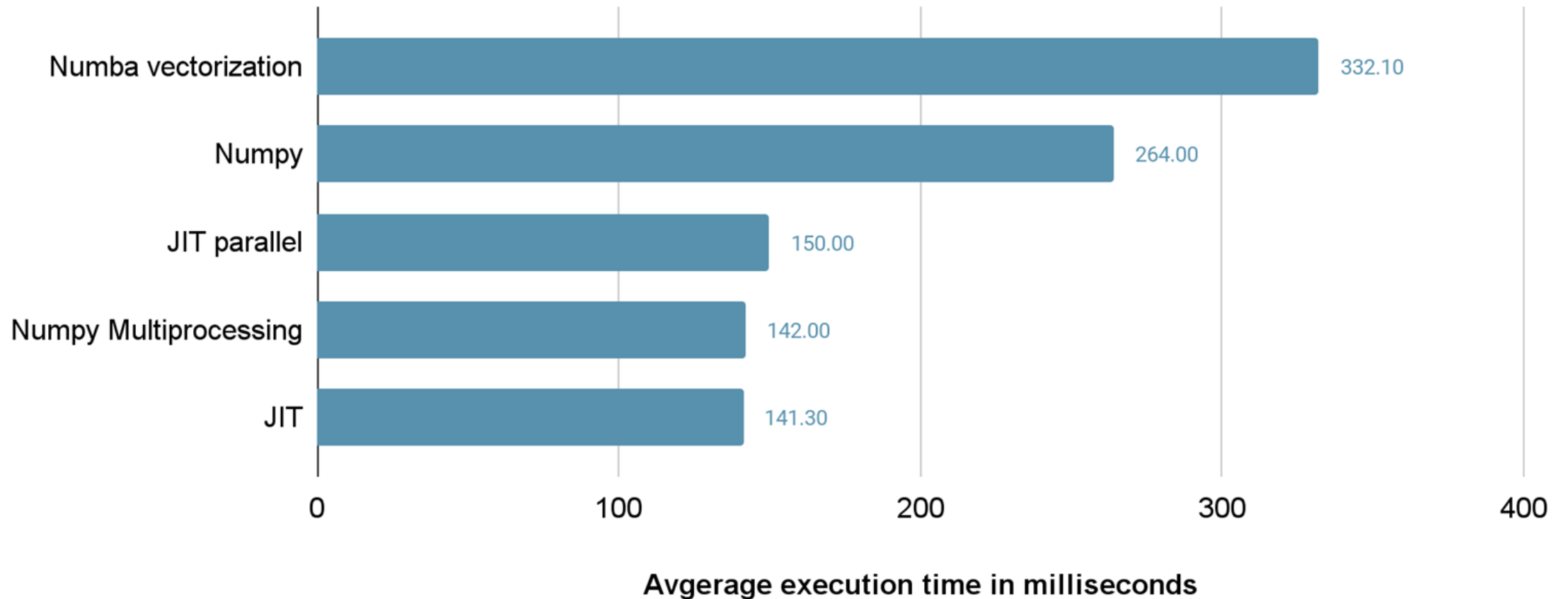
Parallel vector-matrix multiplication

The operation can be easily parallelized through partitioning



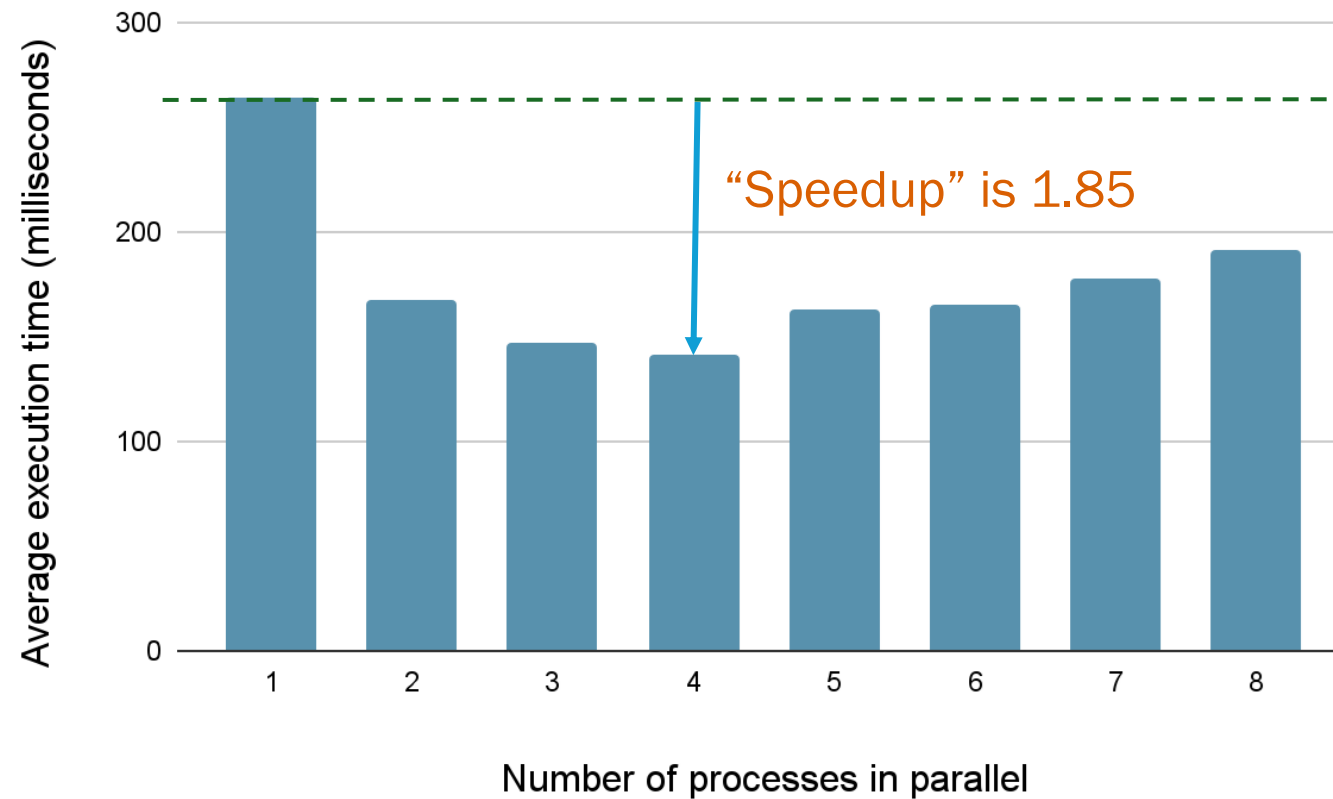
Parallel vector-matrix multiplication

Multiply a vector of length $N = 10\,000$ by an $N \times N$ matrix of integers



Parallel vector-matrix multiplication

Execution time with different number of parallel processes

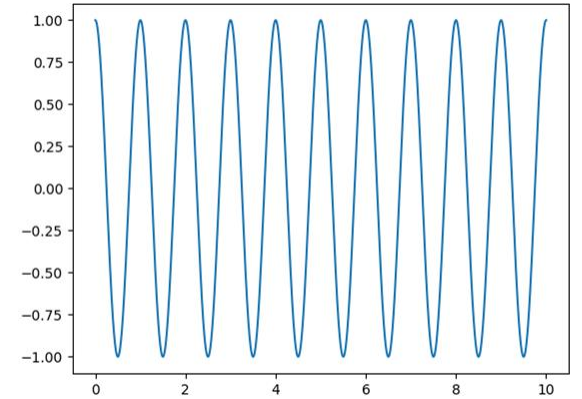
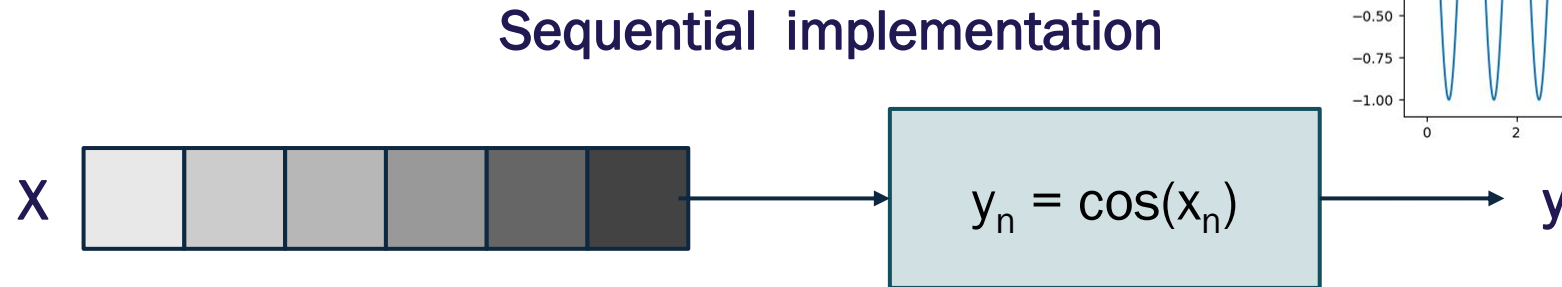


In the CNT job server,
the “speedup” with 12
parallel processes was
3.34

Parallelization overhead

Some codes may run slower in parallel

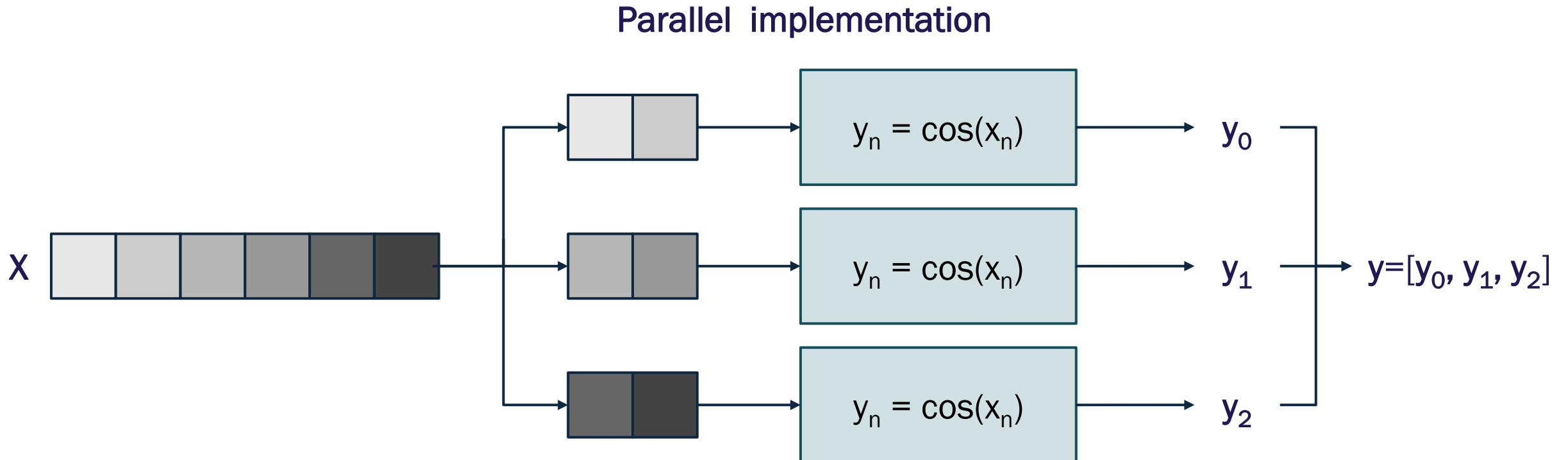
Example: Calculate $y = \cos(x)$ for $x = [x_0, x_1, \dots, x_N]$



Parallelization overhead

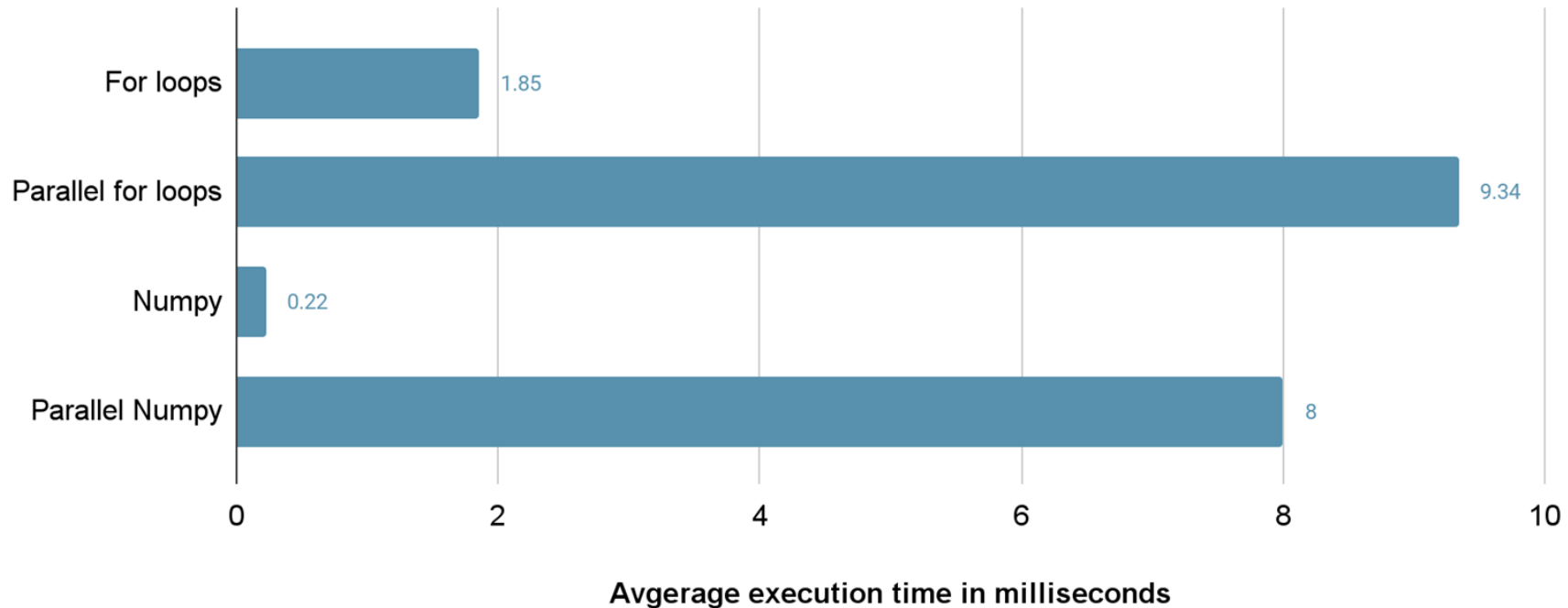
Some codes may run slower in parallel

Example: Calculate $y = \cos(x)$ for $x = [x_0, x_1, \dots, x_N]$



Parallelization overhead

Calculate the value of a cosine function for 5000 points



5. Summary and exercises

Summary

Parallel execution of tasks can be done via threads and processes

Threads are separate flows of execution, connected to a main thread

A process may consist of one or more threads

Communication between processes must be explicit

Creating, managing, and communicating among parallel tasks introduces overhead

Parallel execution does not guarantee a faster execution

Combine parallel execution with vectorization for best results

Exercises

1. Speeding up the calculations: matrix multiplication $C = A \times B$

Try different methods for parallelization (programming approach) and for data partitioning to multiply two square matrices **A** and **B** with $n \times n$ elements, where $n=10000$.

- a) Discuss which method for partitioning could offer better parallelization opportunities
- b) Select a method for vectorization within each parallel process
- c) Compare your results with the execution time using
 - Simple for loops (3 are needed here)
 - Numpy matmul

Types of partitioning in the next pages

Exercises

2. Optimal number of threads/processes

Select an approach to approximate π from the two we covered in last lecture:

1. Generate random points in a squared area
2. Riemann integral of sinc function

Implement a code for parallel execution and plot the execution time as a function of the number of processes.

- a) What is the optimal number of processes for your platform (PC)?
- b) How does this relate to the number of cores in your platform?
- c) Do you observe different results with different platforms?

Exercises

3. A smart metering simulation using threads or processes

A system has 100 sensors that activate at the same time

The temperature is a random number. Use “`2 + np.random.randn()`” to simulate

All the sensors read the temperature at the same time and attempt to transmit

An access point must keep track of the temperature transmitted by each sensor.

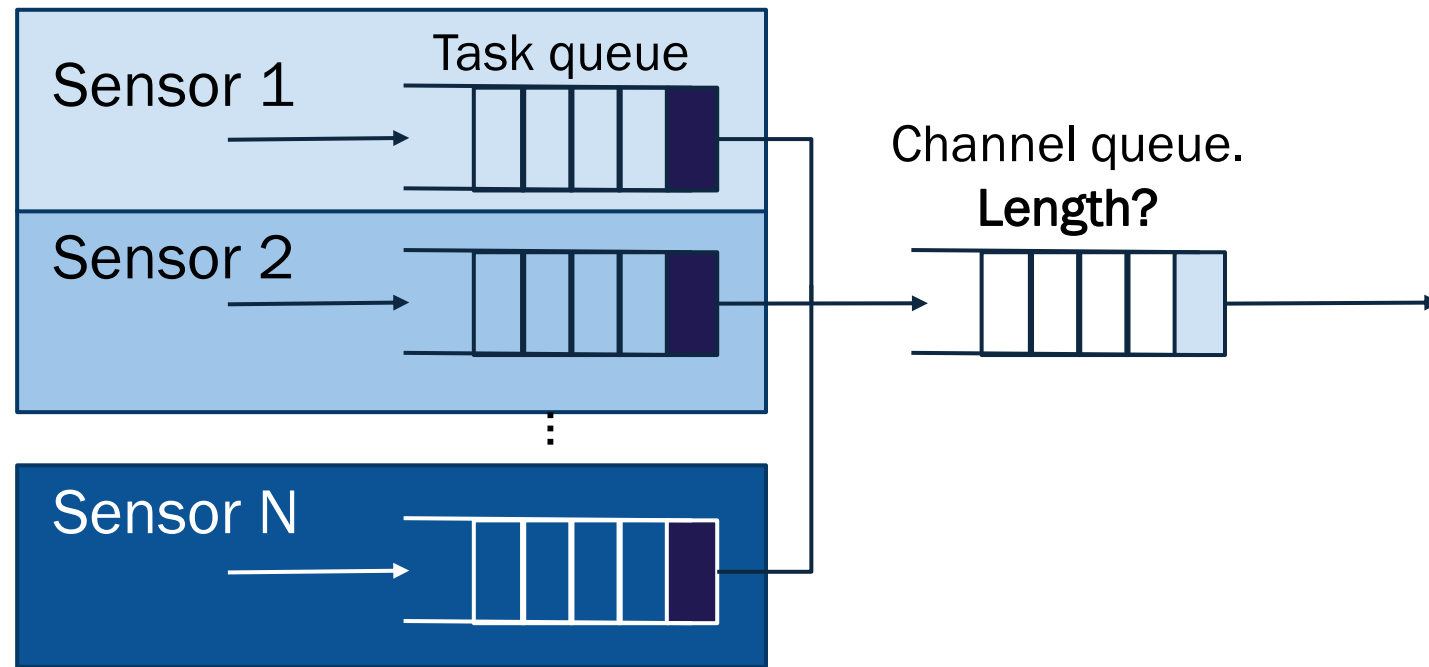
- a) Should you use threading or multiprocessing?
- b) Which synchronization primitives should you use? Why?
- c) How long does it take to record the temperatures?
- d) What is the order of transmission of the sensors?

Tips: Producer-consumer queues in smart metering exercise

The **sensors generate packets** and assign them to a channel queue. What is its length?

Are the sensors synchronized by the Base Station, or is it pure competition among them?

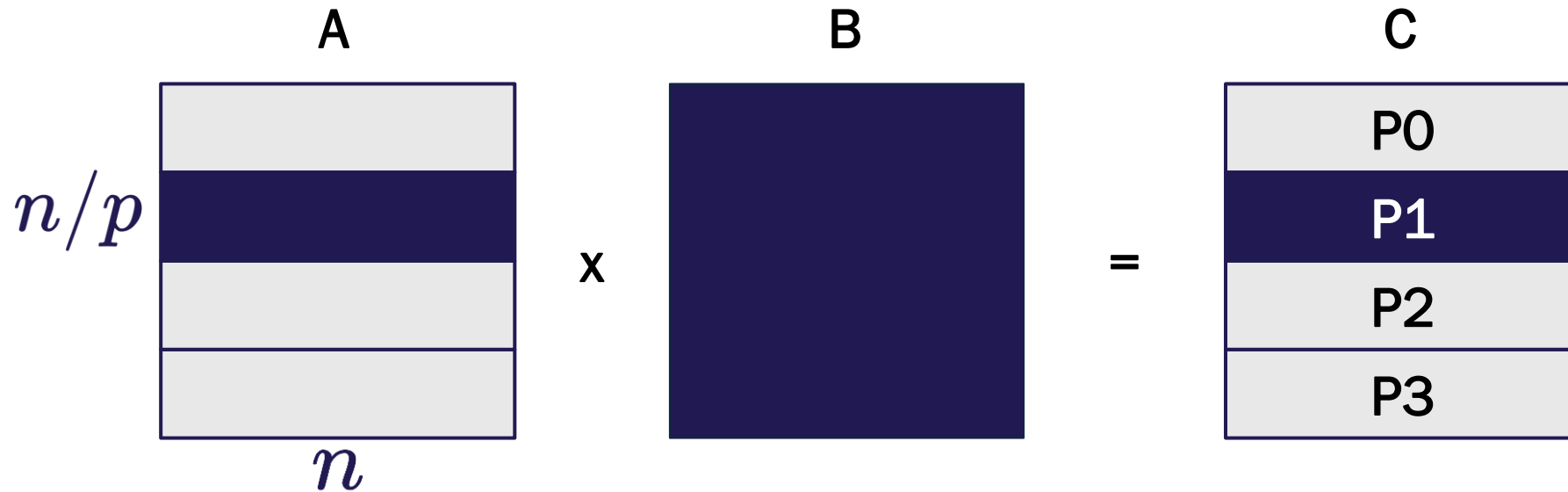
Example: How long should the sensors wait to attempt access again?



Appendix: Matrix partitioning methods

One-dimensional partitioning

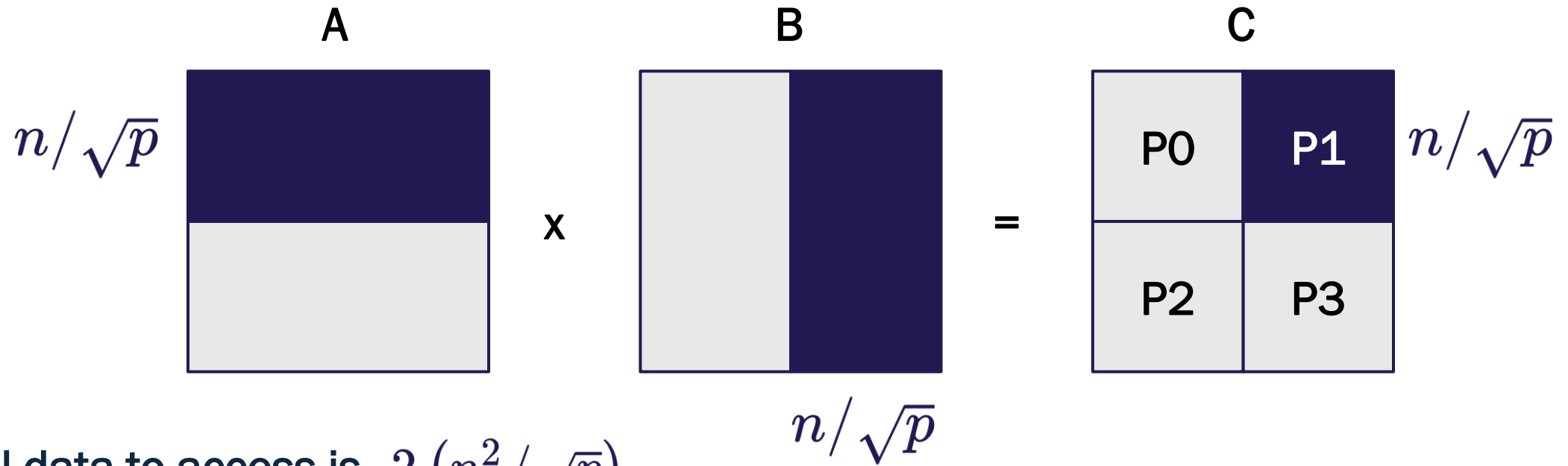
One-dimensional partition of the output matrix **C**. Each process **P_m** needs access to **B**



Total data to access is $n^2/p + n^2$

Two-dimensional partitioning

Two-dimensional partition of the output matrix **C**.



Total data to access is $2 (n^2 / \sqrt{p})$

Decomposition for matrix multiplication

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} = \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Decompose into 12 tasks: from 1 $n \times n$ matrix to 4 $n/2 \times n/2$ sub matrices

$$C_{1,1} = A_{1,1} B_{1,1} + A_{1,2} B_{2,1}$$

$$\mathcal{O}(n^3) \rightarrow \mathcal{O}(n^3/8)$$

$$C_{1,2} = A_{1,1} B_{1,2} + A_{1,2} B_{2,2}$$

$$C_{2,1} = A_{2,1} B_{1,1} + A_{2,2} B_{2,1}$$

$$C_{2,2} = A_{2,1} B_{1,2} + A_{2,2} B_{2,2}$$

That's it for today